RECEIPT SCANNER

VOLUME 2

SOFTWARE DESIGN DOCUMENT

RECEIPT SCANNER

28 MARCH 2026

Author: James Green

Project: Receipt Scanner V2

Document Class: SWDD

Version: 0.1.0

THIS PAGE INTENTIONALLY LEFT BLANK

RECEIPT SCANNER – SOFTWARE DESIGN DOCUMENT

This document and all associated work are the intellectual property of James Green and are maintained under the domain ckplace.org. Unauthorized reproduction, distribution, or use of this material, in whole or in part, without explicit permission is prohibited.

This work has been completed under the direction of Dr. Natalia Bell, Marymount University, as part of the coursework for IT-548-A, and is submitted in fulfillment of academic requirements for the specified class.

James Green, 2024.

THIS PAGE INTENTIONALLY LEFT BLANK

TABLE OF CONTENTS

RECEIPT SCANNER I

TABLE OF CONTENTS III
LIST OF TABLES V
LIST OF FIGURES V
CHAPTER 1 INTRODUCTION 1-1
1.1 Purpose *1-1*
1.2 Scope *1-1*
1.3 Document Overview *1-2*
CHAPTER 2 SYSTEM OVERVIEW 2-1
2.1 System Perspective *2-1*
2.2 System Functions *2-1*
2.3 User Characteristics *2-2*
2.4 Operating Environment *2-2*
2.5 Constraints *2-2*
2.6 Assumptions and Dependencies *2-3*
CHAPTER 3 DESIGN CONSIDERATIONS 3-1
3.1 Assumptions *3-1*
3.2 Constraints *3-1*
3.3 Design Goals *3-2*
3.4 Design Rationale *3-2*
3.5 Tradeoffs *3-3*
CHAPTER 4 SYSTEM ARCHITECTURE 4-1
4.1 Architectural Overview *4-1*
4.2 Architectural Style *4-2*
4.3 System Decomposition *4-3*
4.4 Component Interaction *4-4*
4.5 Data Flow *4-4*
4.6 Design Decomposition *4-5*
4.7 Architectural Constraints *4-5*
CHAPTER 5 DETAILED DESIGN 5-1
5.1 Overview *5-1*
5.2 Receipt Processing Controller *5-1*
5.3 Input Validation Component *5-2*
5.4 Image Preprocessing Component *5-3*
5.5 OCR Processing Component *5-4*
5.6 Text Parsing Component *5-5*
5.7 Data Management Component *5-6*
5.8 Aggregation and Summary Component *5-6*
5.9 Export Component *5-7*
5.10 User Interaction Support Component *5-7*
5.11 Processing Logic *5-8*
5.12 Design Considerations for Maintainability and Extension *5-8*
CHAPTER 6 DATA DESIGN 6-1

6.1 Overview *6-1*
6.2 Data Entities *6-1*
6.3 Data Relationships *6-2*
6.4 Data Storage Design *6-3*
6.5 Data Integrity and Validation *6-3*
6.6 Data Lifecycle *6-4*
CHAPTER 7 INTERFACE DESIGN 7-1
7.1 Overview *7-1*
7.2 User Interface Design *7-1*
7.3 System Interfaces *7-3*
7.4 Interface Constraints *7-4*
CHAPTER 8 ERROR HANDLING 8-1
8.1 Input Errors *8-1*
8.2 Fault Tolerance *8-2*
8.3 Recovery Mechanisms *8-2*

APPENDIX A A-1

APPENDIX B B-1

B.1 DEPLOYMENT STRATEGY B-3
B.2 DISTRIBUTION B-3

LIST OF TABLES

Table 4.3-1 System Decomposition 4-3
Table 6.2-1 - Receipt Data Entity Attributes 6-1
Table 6.2-2 - Monthly Summary Data Entity Attributes 6-2
Table 7.2-1 - Interface Components 7-2

LIST OF FIGURES

Figure A-1 - Main Page 7-3
Figure B-1 - Docker Image B-3

THIS PAGE INTENTIONALLY LEFT BLANK

CHAPTER 1
INTRODUCTION

1.1 Purpose

This document provides a detailed design description of the Receipt Scanner (Version 2) system. It defines the system architecture, component-level design, data structures, and interface specifications necessary to implement the requirements defined in Volume 1 (SRS).

The purpose of this document is to translate system requirements into a structured design that supports implementation, testing, and future system extension.

1.2 Scope

The design described in this document supports a system that:

- Accepts receipt images as input
- Processes images using OCR
- Extracts structured financial data
- Stores, displays, and aggregates receipt data
- Provides export functionality

This design focuses on a lightweight, modular architecture suitable for individual users, consistent with the constraints defined in the SRS (e.g., Python-based, local/web deployment).

1.3 Document Overview

This document presents the Software Design Description (SWDD) for the Receipt Scanner (Version 2) system. It is organized to provide a structured description of the system's design in accordance with established software engineering practices.

Chapter 2 provides a system overview from a design perspective, including the system's structure and major components. Chapter 3 outlines design considerations, including constraints, assumptions, and guiding principles. Chapter 4 defines the system architecture, describing how the system is organized and how components interact. Chapter 5 provides detailed design information for individual system components and processing logic.

Chapter 6 defines the system's data design, including data structures and relationships. Chapter 7 describes the interface design, including user and system interactions. Chapter 8 outlines error handling and fault management strategies to ensure reliable system operation.

CHAPTER 2
SYSTEM OVERVIEW

2.1 System Perspective

The Receipt Scanner (Version 2) system is a standalone application designed to digitize, process, and analyze receipt data for individual users. From a design perspective, the system is structured as a modular processing pipeline that transforms unstructured receipt images into structured financial records. It operates as a self-contained solution, integrating user input, OCR processing, and data management components to support receipt tracking and analysis.

The system extends the functionality described in the SRS by organizing processing logic into distinct components responsible for image preprocessing, text extraction, parsing, data storage, and presentation. This modular structure supports extensibility, allowing additional receipt types and processing logic to be incorporated without requiring significant changes to the overall system design

2.2 System Functions

At a design level, the system provides a set of coordinated functions that implement the requirements defined in the SRS. These functions are realized through the interaction of system components and processing stages.

The system supports the following core functions:

- Receipt image upload and validation
- Selection of receipt type to guide processing logic
- Image preprocessing to prepare input for OCR
- OCR-based extraction of textual data from receipt images
- Parsing of extracted text to identify structured financial fields
- Creation and storage of structured receipt records
- Batch processing of multiple receipts within a single workflow
- Display and user modification of processed receipt data
- Aggregation of receipt data to generate monthly summaries
- Calculation of derived financial metrics such as totals and averages
- Export of receipt data in CSV format

These functions collectively implement the transformation of receipt images into usable financial information, supporting both record-keeping and basic analysis capabilities.

2.3 User Characteristics

The system is intended for individual users with varying levels of technical experience. General users are expected to have minimal technical expertise and rely on a simple and intuitive interface for uploading receipts, reviewing processed data, and generating summaries. Advanced users may utilize exported data for further analysis in external tools and are expected to have familiarity with data handling concepts.

The system is designed to minimize required user interaction during processing while providing clear feedback and the ability to correct extracted data when necessary.

2.4 Operating Environment

The system is designed to operate within a standard computing environment capable of supporting a Python-based application. It may be deployed as a local application or as a lightweight web-based system accessible through a modern web browser.

The environment must support:

- Execution of the Python runtime and required libraries
- File system access for receipt storage and data persistence
- Sufficient processing capability for OCR operations

This design ensures compatibility with general-purpose computing systems without requiring specialized hardware or infrastructure.

2.5 Constraints

The system design is influenced by several constraints identified in the SRS. These include technology constraints associated with the Python-based implementation and reliance on OCR libraries, which affect processing accuracy and performance. Resource constraints limit the use of computationally intensive methods, reinforcing the need for a lightweight and efficient design.

Additionally, the system is constrained by the variability and quality of input receipt images, which may impact the reliability of OCR and parsing processes. Development constraints associated with the academic context also influence the scope and prioritization of system features.

2.6 Assumptions and Dependencies

The system is developed under the assumption that users will provide receipt images of sufficient quality for processing and will correctly identify receipt types when required. It is also assumed that receipt formats contain recognizable patterns that can be processed using defined parsing logic.

Key dependencies include OCR processing libraries, the Python runtime environment, and user interface components that enable interaction with the system. These dependencies define the operational boundaries of the system and must be satisfied for correct system functionality.

THIS PAGE INTENTIONALLY LEFT BLANK

CHAPTER 3
DESIGN CONSIDERATIONS

3.1 Assumptions

The design of the Receipt Scanner (Version 2) system is based on several operational assumptions derived from the system requirements. It is assumed that users will provide receipt images of sufficient quality to enable accurate OCR processing, including adequate resolution, orientation, and minimal visual noise. The system also assumes that users will correctly identify or select the appropriate receipt type when required, allowing the system to apply corresponding parsing logic.

Additionally, it is assumed that receipt formats, while variable, contain consistent structural patterns (such as dates, totals, and vendor identifiers) that can be identified through rule-based or heuristic parsing. The system is intended for use in a non-adversarial context, and therefore does not account for intentionally malformed or malicious inputs designed to disrupt processing.

3.2 Constraints

The system design is subject to several constraints that influence architectural and implementation decisions. The primary constraint is the use of a Python-based implementation, which limits the system to compatible libraries and frameworks while also guiding design choices toward lightweight and modular solutions. The system's reliance on OCR technologies introduces inherent limitations in accuracy and performance, as OCR results are dependent on input quality and engine capabilities.

Resource constraints also affect the design, as the system is intended to operate in environments with limited computational resources. This restricts the use of complex or computationally intensive processing techniques. Additionally, development constraints associated with the academic context impose limitations on system scope and feature complexity. Variability in receipt formats and data quality further constrains the system, requiring flexible parsing approaches and robust error handling mechanisms.

3.3 Design Goals

The system design is guided by several key goals intended to ensure usability, maintainability, and extensibility. A primary goal is modularity, where system functionality is divided into distinct components responsible for preprocessing, OCR, parsing, data management, and presentation. This modular structure supports independent development and future enhancement of individual components.

Extensibility is also a critical goal, allowing the system to support additional receipt types and processing logic without requiring significant architectural changes. Usability is emphasized to ensure that users with minimal technical expertise can effectively interact with the system, supported by clear feedback and the ability to correct extracted data. Reliability is another key objective, requiring the system to handle errors gracefully and maintain consistent operation under normal usage conditions.

3.4 Design Rationale

The architectural and design decisions for the system are based on the need to efficiently transform unstructured receipt images into structured financial data while operating within defined constraints. A pipeline-based processing model is selected to reflect the sequential nature of receipt handling, where each stage (validation, preprocessing, OCR, parsing, and storage) builds upon the output of the previous stage.

A modular design approach is used to isolate system functionality into discrete components, improving maintainability and enabling targeted enhancements. This approach also supports flexibility in replacing or modifying individual components, such as updating OCR engines or refining parsing logic, without requiring changes to the entire system.

The decision to implement the system as a lightweight, optionally web-based application aligns with the requirement for accessibility and ease of deployment. This design minimizes system complexity while ensuring that core functionality—receipt processing, data management, and reporting—can be performed efficiently within standard computing environments.

3.5 Tradeoffs

Several tradeoffs are inherent in the system design. The use of OCR enables automation of data extraction but introduces potential inaccuracies due to variability in input quality and receipt formats. To address this, the system incorporates user review and editing capabilities, balancing automation with human validation.

The choice of a lightweight architecture improves performance and accessibility but limits the use of advanced processing techniques or large-scale data storage solutions. Similarly, the reliance on rule-based parsing provides simplicity and transparency but may reduce flexibility when handling highly variable or complex receipt formats.

These tradeoffs reflect a design approach that prioritizes practicality, usability, and maintainability within the constraints of the system's intended scope and environment.

THIS PAGE INTENTIONALLY LEFT BLANK

CHAPTER 4
SYSTEM ARCHITECTURE

4.1 Architectural Overview

The Receipt Scanner (Version 2) system is designed using a modular, pipeline-oriented architecture that organizes processing into a sequence of well-defined stages. Each stage performs a specific transformation on the input data, beginning with receipt image ingestion and culminating in structured data storage and presentation. This approach reflects the linear nature of receipt processing while enabling clear separation of concerns between components.

The architecture consists of loosely coupled components responsible for input handling, preprocessing, OCR execution, text parsing, data management, and user interaction. These components communicate through defined interfaces and shared data structures, allowing each stage to operate independently while contributing to the overall workflow. The system supports both single-receipt and batch-processing modes within the same architectural framework.

From a deployment perspective, the architecture supports both local execution and lightweight web-based interaction through a Python-based backend. The system is structured to minimize external dependencies while maintaining flexibility for future integration with additional services or interfaces.

4.2 Architectural Style

The system adopts a hybrid architectural style combining pipeline processing and modular component-based design. The pipeline model structures the flow of data through sequential stages, ensuring that each transformation is applied in a controlled and predictable order. This is particularly suited to receipt processing, where each step depends on the output of the previous stage.

In parallel, the system employs a modular design in which each processing stage is implemented as an independent component. These components encapsulate specific functionality, such as image preprocessing or text parsing, and expose clearly defined interfaces. This enables individual components to be modified, replaced, or extended without impacting the overall system structure.

The combination of these styles provides both operational clarity and architectural flexibility. The pipeline ensures efficient data flow, while modularity supports maintainability and extensibility.

4.3 System Decomposition

The system is decomposed into the following primary components:

Table 4.3-1 System Decomposition

User Interface	Responsible for user interaction, including receipt upload, data display, and correction of extracted information.
Input Validation	Ensures that uploaded files meet required format and quality constraints before processing.
Image Preprocessing	Applies transformations such as grayscale conversion and normalization to improve OCR accuracy.
OCR Processing	Extracts textual data from processed receipt images using an OCR engine.
Text Parsing	Interprets extracted text and identifies structured fields such as vendor, date, and total amount.
Data Management	Generates summaries and derived metrics from stored receipt data.
Aggregation and Reporting	Provides functionality to export structured data into CSV format.
Export	Stores structured receipt data, supports retrieval, and manages aggregation operations.

Each component is designed to operate independently while contributing to the overall processing workflow.

4.4 Component Interaction

Component interaction within the system follows a sequential and event-driven pattern aligned with the processing pipeline. When a user uploads a receipt, the User Interface component initiates the workflow by passing the input to the Input Validation component. Upon successful validation, the data is forwarded to the Image Preprocessing component, which prepares the image for OCR processing.

The OCR Processing component extracts textual content and passes the results to the Text Parsing component. The parsing component interprets the text and produces structured data, which is then stored by the Data Management component. Once stored, the data becomes available for display, modification, aggregation, and export.

Interactions between components are governed by defined input and output formats, ensuring consistency and reducing coupling. Error conditions at any stage are propagated back through the system to provide user feedback and enable corrective action.

4.5 Data Flow

The system processes data through a structured flow that transforms raw input into usable information. The primary data flow begins with receipt images provided by the user and progresses through multiple transformation stages.

Initially, raw image data is validated and preprocessed to enhance quality. The processed image is then converted into unstructured text through OCR. This text is subsequently analyzed and parsed into structured data elements representing receipt attributes. These structured records are stored and later used for display, aggregation, and export.

Data flow is unidirectional during processing stages, ensuring clarity and reducing complexity. However, user interaction introduces bidirectional flow, as users may modify extracted data, which is then updated within the system.

4.6 Design Decomposition

At a lower level, the system design decomposes processing functionality into specialized classes and modules. The central coordinating entity (e.g., a ReceiptScanner class) orchestrates the interaction between processing components. This class integrates preprocessing, OCR, and parsing modules, allowing the system to execute the full processing pipeline in a controlled manner.

Supporting modules are implemented as discrete units with clearly defined responsibilities:

- Image preprocessing module handles image transformation operations
- OCR module interfaces with external OCR libraries
- Parsing module applies pattern recognition and extraction logic
- Data module manages storage structures and aggregation logic

This decomposition supports reuse and simplifies testing by enabling individual modules to be validated independently. It also allows feature toggling (e.g., disabling OCR) without altering core system logic.

4.7 Architectural Constraints

The system architecture is subject to several constraints that influence design decisions. The use of a Python-based implementation limits available frameworks and requires compatibility with selected libraries for OCR and data handling. This constraint reinforces the need for a lightweight and efficient architecture.

Dependence on OCR technology introduces variability in system accuracy, as results are influenced by input quality and OCR engine performance. This constraint necessitates the inclusion of preprocessing steps and user validation mechanisms within the architecture.

Resource limitations further restrict the use of computationally intensive processing techniques, requiring the system to prioritize efficiency and simplicity. Additionally, the system is designed for a non-distributed environment, which limits scalability but simplifies deployment and maintenance.

These constraints define the operational boundaries of the architecture and guide tradeoffs between performance, accuracy, and system complexity.

THIS PAGE INTENTIONALLY LEFT BLANK

CHAPTER 5
DETAILED DESIGN

5.1 Overview

This chapter provides the detailed design description for the internal components of the Receipt Scanner (Version 2) system. While Chapter 4 defines the system at the architectural level, this chapter refines that view by describing the responsibilities, internal behavior, processing logic, and interactions of the primary system components. The intent is to provide a design description sufficient to guide implementation, support verification activities, and enable future maintenance or enhancement.

The detailed design is organized by functional component. Each subsection describes the purpose of the component, the operations it performs, the data it consumes and produces, and its relationship to the larger system workflow. Together, these descriptions define how the system implements the requirements specified in the SRS and the architecture established in the preceding chapter.

5.2 Receipt Processing Controller

The Receipt Processing Controller is the central coordinating element of the system. Its purpose is to manage the end-to-end workflow associated with receipt processing by invoking the appropriate components in the correct sequence. Rather than performing all processing directly, the controller acts as an orchestration layer that receives input, initiates validation, invokes image preprocessing and OCR operations, triggers text parsing, and passes structured results to storage and presentation components.

The controller is responsible for preserving the logical order of operations and ensuring that each processing stage receives the required input from the previous stage. It also manages processing status and determines whether a receipt should continue through the pipeline or be rejected due to errors or invalid input conditions. In a batch-processing scenario, the controller applies the same workflow iteratively across multiple uploaded receipts while preserving per-item status information. This design centralizes workflow logic and reduces coupling between lower-level components.

5.3 Input Validation Component

The Input Validation Component is responsible for ensuring that receipt files satisfy minimum requirements before they enter the processing pipeline. This includes verifying that a file exists, that its type is supported by the system, and that it is suitable for downstream image processing and OCR activities. The component acts as the first internal safeguard against invalid or unusable input and reduces the likelihood of failures later in the workflow.

The validation logic is intended to identify issues such as unsupported file extensions, inaccessible files, missing uploads, or inputs that cannot reasonably be processed as receipt images. When validation fails, the component returns a failure condition and an associated status message that can be propagated to the user interface. When validation succeeds, the receipt is passed to the preprocessing stage. The design emphasizes early detection of errors so that invalid data is rejected before unnecessary processing resources are consumed.

The component may also support limited metadata checks, such as confirming that a selected receipt type is present when required by the workflow. This ensures that any type-specific parsing logic later in the pipeline has the necessary context.

5.4 Image Preprocessing Component

The Image Preprocessing Component is responsible for transforming uploaded receipt images into a form better suited for OCR operations. Because OCR accuracy is heavily dependent on input quality, this component performs enhancement steps designed to improve text visibility and reduce noise prior to extraction. The preprocessing stage is a critical intermediary between raw image input and textual interpretation.

Typical preprocessing operations include grayscale conversion, normalization, and other lightweight image adjustments intended to improve contrast and readability. The component receives a validated receipt image, applies the relevant transformations, and produces a processed image artifact for subsequent OCR use. The design assumes that preprocessing should remain computationally lightweight and deterministic in order to support performance and maintainability goals.

The component should preserve traceability between the original and processed image by using consistent naming or storage conventions. This supports debugging, user preview functionality, and later verification if the extracted results need to be compared to the source image. If preprocessing cannot be completed successfully, the component returns an error state so that the receipt can be flagged appropriately rather than silently producing unusable downstream output.

5.5 OCR Processing Component

The OCR Processing Component is responsible for converting the processed receipt image into machine-readable text. This component serves as the bridge between image-based input and the rule-based extraction logic used later in the system. It receives a preprocessed image and invokes the configured OCR engine to identify and return text content present in the receipt.

Because OCR output may vary in completeness and accuracy based on image quality and receipt formatting, the component is designed to return raw extracted text rather than attempting to interpret it directly. This keeps the OCR function narrowly scoped and separates text extraction from semantic analysis. The resulting text may also be logged or stored in an intermediate form to support debugging, manual review, or future refinement of parsing logic.

The design may permit OCR to be enabled or disabled depending on deployment conditions or demonstration requirements. This supports environments in which the OCR library is unavailable or intentionally omitted. In such cases, the system architecture remains intact, but OCR-dependent processing paths may be bypassed or replaced with mock behavior. This improves flexibility for development, testing, and limited-functionality deployments.

5.6 Text Parsing Component

The Text Parsing Component is responsible for interpreting OCR-extracted text and converting it into structured receipt data. This component applies rule-based and heuristic logic to identify key receipt fields such as vendor name, transaction date, total amount, and any additional attributes relevant to supported receipt categories. It is the principal component through which unstructured text is transformed into application-usable financial information.

The design assumes that receipt text contains recurring patterns that can be recognized through defined parsing logic. Vendor identification may be supported through a known vendor list or similar recognition mechanism, while numeric and date fields may be extracted through pattern matching rules. Because OCR text may contain noise, ambiguity, or omissions, the parser must be tolerant of imperfect input and capable of returning partial results where complete extraction is not possible.

The parser should distinguish between successfully extracted values, unknown values, and values that require user review. This distinction is important because it supports the broader system goal of semi-automated processing rather than fully autonomous interpretation. When sufficient information is available, the parser constructs a structured receipt record and forwards it to the data management layer. When uncertainty remains, the record may still be created but flagged for correction or manual validation.

5.7 Data Management Component

The Data Management Component is responsible for storing, updating, retrieving, and organizing structured receipt records generated by the processing pipeline. Once the parsing component has created a receipt record, this component maintains that record as part of the system's working data set and makes it available to the user interface, aggregation logic, and export functions.

The design supports structured storage of receipt attributes such as file identifier, vendor, date, total amount, status, and any supporting metadata needed for display or review. The component must also support modification of stored values when a user corrects OCR or parsing errors through the interface. For this reason, storage operations include both record creation and record update behavior.

In a lightweight deployment, the data management layer may rely on in-memory structures, file-based storage, or other simple persistence mechanisms consistent with the system's constraints. The design does not require enterprise-scale database infrastructure, but it does require that stored records be internally consistent and retrievable for summary generation and export. The component therefore serves as the persistence and state-management foundation of the application.

5.8 Aggregation and Summary Component

The Aggregation and Summary Component is responsible for producing higher-level views of stored receipt data. While individual receipt records support transaction-level tracking, this component derives broader financial information by grouping and summarizing data across multiple records. Its purpose is to support reporting features such as monthly summaries and other basic analytical outputs described in the system requirements.

The component operates on structured receipt data maintained by the Data Management Component. It may group receipts by time period, such as month, and compute totals, subtotals, averages, or counts as required by the application. Derived values produced by this component are then made available for presentation in the user interface or for inclusion in exported reports.

This design intentionally keeps aggregation logic separate from both parsing and storage logic. By isolating summary generation into its own component, the system improves maintainability and allows reporting logic to evolve independently from the receipt-processing pipeline.

5.9 Export Component

The Export Component is responsible for transforming structured receipt data and associated summary data into an external representation suitable for user download or offline use. In the current system scope, the primary export format is Comma-Separated Values (CSV), which provides a simple and widely compatible mechanism for data transfer to spreadsheets and other analysis tools.

The component receives structured records from the data management layer and formats them according to a defined column structure. Exported data must preserve field consistency, use stable ordering, and represent records in a manner suitable for later analysis. Where summaries are included, the export logic should clearly separate transaction-level data from aggregated outputs or provide distinct export structures as needed.

By isolating export behavior into a dedicated component, the system allows future support for additional export formats without changing the internal processing logic. This design also improves testability by separating data formatting concerns from storage and presentation concerns.

5.10 User Interaction Support Component

The User Interaction Support Component encompasses the backend logic required to support user review, correction, and display of processed receipt information. Although the visible interface is addressed more fully in the later interface-design chapter, this component exists at the detailed design level because it defines how processed records are prepared for display and how user-submitted corrections are accepted and applied.

This component is responsible for presenting processing status, distinguishing between valid, broken, and correction-required records, and ensuring that only appropriate records are exposed through interactive links or views. It also supports preview behavior for processed images and structured display of extracted receipt fields. When a user modifies a record, the component forwards those updates to the data management layer so that corrected data becomes the authoritative version of the record.

The design goal is to ensure that the system remains usable even when OCR or parsing is imperfect. Rather than assuming fully accurate automation, the interaction support logic reinforces a human-in-the-loop model in which extracted values can be reviewed and corrected before they are used in summaries or exports.

5.11 Processing Logic

The detailed processing logic of the system follows a defined sequence. First, the user submits one or more receipt images through the interface. The controller forwards each receipt to the validation component, which confirms that the file is processable. If validation succeeds, the image is passed to preprocessing, where it is converted into a form more suitable for OCR. The OCR component then extracts raw text from the processed image.

Next, the parsing component analyzes the extracted text and attempts to build a structured receipt record. That record is stored by the data management component and made available for user review. If the user identifies an error or omission, the corrected values are submitted back to the system and replace the original extracted values. Finally, the stored records can be aggregated into summaries or exported in CSV format.

This processing logic reflects the pipeline architecture described earlier while also incorporating controlled user intervention. The result is a semi-automated design that supports efficient receipt handling without eliminating user oversight.

5.12 Design Considerations for Maintainability and Extension

The detailed design of the system emphasizes maintainability through modular separation of responsibilities. Each major function of the system is implemented within a distinct component boundary, making it possible to revise OCR behavior, parsing rules, or export logic without redesigning the entire application. This supports the system's long-term viability and reduces the cost of future enhancements.

The design also supports extensibility. Additional receipt categories, expanded parsing rules, alternative OCR backends, or new reporting formats can be introduced by modifying or extending individual components rather than altering the core workflow. This is especially important for a system intended to evolve over time as new requirements emerge or as receipt formats become more varied.

CHAPTER 6
DATA DESIGN

6.1 Overview

This chapter defines the data structures, formats, and relationships used within the Receipt Scanner (Version 2) system. The purpose of this chapter is to describe how information is represented, stored, and manipulated throughout the system lifecycle. While previous chapters define how the system processes data, this chapter defines the structure and organization of that data.

The data design supports the transformation of unstructured receipt images into structured financial records. It also enables aggregation, user correction, and export operations. The design prioritizes simplicity, consistency, and compatibility with lightweight storage mechanisms.

6.2 Data Entities

6.2.1 *Receipt*

The Receipt entity represents a single processed transaction derived from an input receipt image. It serves as the primary data unit within the system.

Table 6.2-1 - Receipt Data Entity Attributes

Receipt ID (unique identifier)
Filename (original uploaded file reference)
Merchant
Transaction Date
Total Amount
Category
Status (valid, broken, etc)
Raw OCR Text

This entity supports both automated extraction and user-modified values.

6.2.2 *Receipt Collection*

The Receipt Collection represents a group of Receipt entities processed within a session or stored within the system.

Characteristics:

Maintains a mapping of receipt identifiers to receipt records

Supports batch operations such as aggregation and export

Enables filtering and retrieval for display or reporting

6.2.3 *Monthly Summary*

The Monthly Summary entity represents aggregated financial data derived from multiple receipts over a defined time period.

Table 6.2-2 - Monthly Summary Data Entity Attributes

Month/Year Identifier
Total Expenditure
Number of Receipts
Average Transaction Value
Category Based Totals

This entity is derived rather than directly stored and is generated dynamically.

6.3 Data Relationships

The system maintains simple hierarchical relationships between entities:

A **Receipt Collection** contains multiple **Receipt entities**

A **Monthly Summary** is derived from one or more **Receipt entities** grouped by date

These relationships are intentionally non-relational and lightweight to support simple storage mechanisms. Referential integrity is maintained through internal identifiers rather than enforced database constraints.

6.4 Data Storage Design

The system uses a lightweight storage approach consistent with its design constraints. Data persistence may be implemented using:

- In-memory data structures for active sessions
- File-based storage for receipt images and extracted text
- Structured serialization (e.g., JSON-like formats) for receipt records

The design avoids reliance on complex database systems in favor of simplicity and portability. Stored data must remain consistent across sessions where persistence is required.

6.5 Data Integrity and Validation

Data integrity is maintained through validation at multiple stages:

- Input validation ensures only valid files enter the system
- Parsing validation ensures extracted fields meet expected formats
- User correction ensures final stored values are accurate

The system distinguishes between:

- Valid data
- Incomplete data
- Invalid or “broken” records

This classification supports safe downstream use of data in aggregation and export operations.

6.6 Data Lifecycle

The lifecycle of receipt data follows a defined progression:

1. **Creation:** Receipt image is uploaded
2. **Transformation:** Image is processed and converted to text
3. **Structuring:** Text is parsed into a receipt record
4. **Storage:** Record is saved in the data management system
5. **Modification:** User may correct or update fields
6. **Aggregation:** Data contributes to summaries
7. **Export:** Data is formatted for external use

This lifecycle ensures traceability from raw input to final output.

CHAPTER 7
INTERFACE DESIGN

7.1 Overview

This chapter defines the interfaces through which users and system components interact. It includes both user-facing interfaces and internal system interfaces. The goal is to ensure consistent interaction patterns, usability, and clear communication between components.

7.2 User Interface Design

7.2.1 *General Characteristics*

The user interface is designed to be simple and accessible, requiring minimal technical expertise. It provides functionality for uploading receipts, reviewing extracted data, correcting errors, and generating summaries.

The interface prioritizes clarity and feedback, ensuring that users understand processing results and system status at all times.

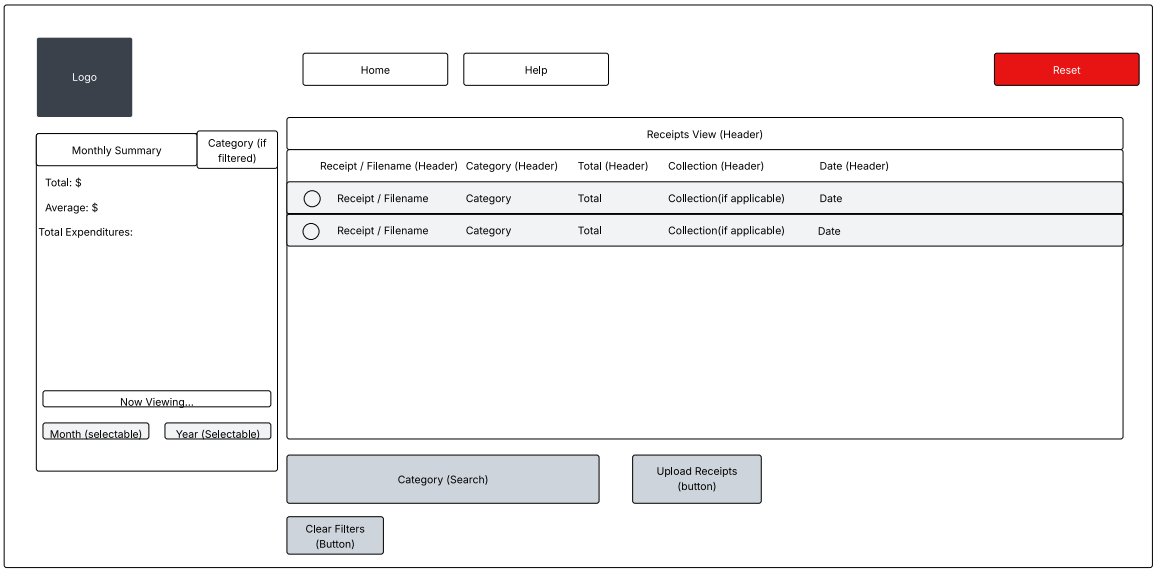
7.2.2 *Interface Components*

Table 7.2-1 - Interface Components	
Interface	**Function(s)**
Receipt Upload	- Allows users to upload one or more receipt images. - Displays validation feedback for invalid files.
Results Display	- Shows extracted receipt data in structured form. - Differentiates between valid, broken, and editable records.
Correction Interface	- Enables users to modify extracted values. - Updates stored records upon submission.
Summary Interface	- Displays aggregated financial data. - Supports basic filtering (e.g., by month).
Export Interface	Provides CSV download functionality.

7.2.3 *Wireframes*

Page Wireframe

Figure A-1 - Main Page



7.3 System Interfaces

7.3.1 *Internal Interfaces*

Internal interfaces define how system components communicate.

Examples include:

- Preprocessor → OCR input/output format
- OCR → Parser text interface
- Parser → Data Management structured record format

These interfaces use well-defined data structures to ensure compatibility and reduce coupling.

7.3.2 *External Interfaces*

File System Interface

Used for storing uploaded images and processed outputs

OCR Library Interface

- Provides text extraction functionality
- Invoked through a defined API or wrapper

CSV Export Interface

Outputs structured data in a standardized format

7.4 Interface Constraints

Interface design is constrained by:

- Simplicity requirements for non-technical users
- Limitations of the chosen web framework
- Dependency on OCR library input/output formats

These constraints influence both usability and implementation choices.

CHAPTER 8
ERROR HANDLING

8.1 Input Errors

8.1.1 *Input Errors*

Invalid inputs are rejected early in the pipeline. The system provides clear feedback indicating the reason for rejection.

8.1.2 *Processing Errors*

If a processing stage fails:

- The receipt is marked as “broken”
- Processing is halted for that item
- Other receipts in batch processing continue unaffected

8.1.3 *Parsing Errors*

If parsing produces incomplete or uncertain results:

- The record is flagged for user correction
- Partial data is preserved
- The system avoids discarding usable information

8.1.4 *System Errors*

Unexpected failures are handled through:

- Graceful degradation of functionality
- User notification where appropriate

8.2 Fault Tolerance

The system is designed to tolerate partial failures:

- Batch processing isolates errors per receipt
- Failed components do not halt the entire system
- User intervention compensates for automation limitations

This ensures continuity of operation under normal usage conditions.

8.3 Recovery Mechanisms

Recovery mechanisms include:

- Allowing users to reprocess receipts
- Enabling manual correction of extracted data
- Preserving original input data for re-evaluation

These mechanisms ensure that errors do not result in permanent data loss.

APPENDIX A

THIS PAGE INTENTIONALLY LEFT BLANK

NOMENCLATURE

CSV	Comma-Separated Values
FR	Functional Requirement
Monthly Summary	Aggregated financial data grouped by month
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
Receipt	A structured record representing a single transaction
Receipt Scanner	The system described in this document
RTM	Requirements Tracability Matrix
SRS	Software Requirement Specification
TC	Test Case
UI	User Interface

THIS PAGE INTENTIONALLY LEFT BLANK

APPENDIX B

THIS PAGE INTENTIONALLY LEFT BLANK

B.1 Deployment Strategy

Receipt Scanner (version 2) is deployed using a live webserver configured on Render.com. To package its deployable environment, ReceiptScanner utilizes the below docker image.

Figure B-1 - Docker Image

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
  PYTHONUNBUFFERED=1

RUN apt-get update && apt-get install -y --no-install-recommends \
  tesseract-ocr \
  libtesseract-dev \
  libleptonica-dev \
  && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN pip install --no-cache-dir -e .

EXPOSE 8000
CMD ["python", "wsgi.py"]
```

B.2 Distribution

Receipt Scanner hosts its documentation on https://docs.ckplace.org/ReceiptScanner-V1

THIS PAGE INTENTIONALLY LEFT BLANK

END OF DOCUMENT